MDN CORE MODULES EĞİTİM REHBERİ

JavaScript & JavaScript'in Kalbi: Event Loop ve Asenkron Mimari

Sunan: **Ayşe Nurcihan Akbaş** | Grup: **Altay**

```
// altay-sunum.js — Başlangıç yapılandırması
const sunum = {
  ders:    "Web Programlama I",
  grup:    "Altay",
  moduller: [
    "1: Dinamik Betik Yazımı",
    "2: Çerçeveler ve Kütüphaneler",
    "3: Erişilebilirlik (A11y)"
  ]
};

// Sunumu başlat
console.log("Başlıyoruz!");
```

Uyarı: SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında eğitim amaçlı derlenmiştir. Kaynak: MDN Web Docs (CC-BY-SA).

YOL HARİTASI

İçindekiler

MDN Çekirdek
Modüllerinin son 3 başlığı

MDN CORE MODULES HARİTASI

Bugün Ne Öğreneceğiz?

MODÜL 1

JavaScript ile Dinamik Betik Yazımı

- **Tanım & Tarihçe:** JS nedir, nasıl çalışır?
- **Temeller:** Değişken, sayı, dize, dizi
- **Akış:** Koşullar, döngüler, fonksiyonlar
- **Etkileşim:** Nesneler, DOM, Olaylar
- **Asenkron:** Event Loop, Fetch, JSON, Hata

MODÜL 2

JavaScript Çerçeveleri ve Kütüphaneleri

- **Neden Çerçeve:** DOM zorluğu ve 4 büyükler
- **Ana Özellikler:** JSX, Sanal DOM, Mimari
- **React Temelleri:** Bileşen (Component) ve Props
- **Etkileşim:** State (useState) ve Olaylar
- **Odak Yönetimi:** Koşullu işleme ve useRef

MODÜL 3

Erişilebilirlik (Accessibility)

- **A11y Nedir:** Engel türleri ve WCAG
- **Semantik HTML:** Doğru etiket ve klavye
- **CSS & JS A11y:** Kontrast ve odak yönetimi
- **WAI-ARIA:** Rol, durum ve canlı bölgeler
- **Multimedya & Mobil:** Ekran okuyucu testi

BENZETME Modül 1 evin elektriğini döşemek, Modül 2 akıllı ev sistemi kurmak, Modül 3 ise eve engelli rampası ve sesli asansör eklemektir.

WEB TEKNOLOJİLERİ PASTASININ 3. KATMANI

JavaScript Nedir ve Sayfaya Nasıl Eklenir?

1. Web'in Programlama Dili

Statik HTML ve CSS sayfasını dinamik, canlı ve kullanıcıyla etkileşimli hale getiren betik dilidir.

2. Çekirdek Dil + API Gücü

DOM, Fetch ve Geolocation gibi Tarayıcı API'leri hazır mobilya kiti gibi ekstra süper güçler sunar.

3. Güvenli Yürütme Ortamı

Her sekme ayrı bir kovada çalışır; diğer sitelerin verilerine izinsiz erişemez ve kodu yukarıdan aşağıya okur.

```
<script type="module" src="script.js">script>
<script src="app.js" defer>script>
```

Neden defer veya module?

HTML henüz yüklenmeden JS çalışırsa olmayan öğeyi seçmeye çalışır ve hata verir. `defer` HTML bitene kadar bekletir.

BENZETME HTML binanın iskeleti, CSS boyası, JavaScript ise elektriğidir: elektrik olmazsa bina durur ama hiçbir ışık yanmaz ve asansör çalışmaz.

1995'TEN BUGÜNE EVRİM

JavaScript'in Kısa Tarihi ve Temel Özellikleri

1995
Brendan Eich 10 günde yazar: Mocha → LiveScript → JavaScript.

1997
ECMAScript standardı doğar; tüm tarayıcılar ortak kuralları izler.

2009
Node.js ile sunucuya taşınır; tek dille ön ve arka yüz yazılır.

2015+
ES6 devrimi: `let`, `const`, ok fonksiyonu, Promise, async/await.

Yorumlanan / JIT Derlemeli
V8 motoru kodu çalışırken anında makine koduna çevirir (Just-In-Time).

Tek İş Parçacıklı (Single Thread)
Tek bir Call Stack vardır; aynı anda yalnızca tek bir kod satırı işlenir.

Dinamik Tipli & Olay Güdümlü
Değişken tipi çalışırken belli olur; tıklama ve ağ olaylarına tepki verir.

BENZETME Java ile JavaScript'in akrabalığı yoktur (halı ile halıfleks gibi). Tek iş parçacığı ise tek kasalı market gibidir: müşteriler sırayla geçer.

BİLGİYİ SAKLAMAK: LET, CONST VE VERİ TİPLERİ

Değişkenler, Sabitler ve Dinamik Tipleme

```
// degiskenler.js — Tanımlama ve veri tipleri
const okul    = "SCÜ UBYO";          // Sabit dize (değişmez)
let   puan    = 85;                  // Sayı (güncellenebilir)
      puan    = 95;                  // Yeni değer atandı
let   aktifMi = true;                // Boolean (Mantıksal)
const diller  = ["HTML", "CSS", "JS"]; // Dizi (Array)
const ders    = { kod: "WEB101" };   // Nesne (Object)

typeof puan;   // "number"
typeof okul;   // "string"
```

const vs let vs var

Yapabildiğinde `const`, değer değişecekse `let` kullan. Eski `var` kapsam (hoisting) hatası üretir.

Kutu Yok vs Boş Kutu

Tanımsız değişken `ReferenceError` verir (kutu yok); değer atanmamış `let x;` ise `undefined` döner (boş kutu).

Adlandırma Kuralı

`lowerCamelCase` (`kullaniciAdi`) kullanılır; sayıyla başlamaz.

BENZETME Değişken değerin kendisi değil, üstü etiketli bir karton kutudur: `let` kutunun içindekini değiştirmeye izin verir, `const` kutuyu mühürler.

TEMEL MATEMATİK VE KARŞILAŞTIRMA

Sayılar, Aritmetik ve Katı Eşitlik (===)

```
// 1. Dizeyi sayıya çevirme tuzağı
let girdi = "74";
girdi + 3;           // "743" (Metin birleştirir!)
Number(girdi) + 3;   // 77    (Gerçek toplama)

// 2. Aritmetik ve Atama
let sayac = 10;
sayac += 5;          // 15    (sayac = sayac + 5)
sayac++;             // 16    (1 artırır)
(1.766).toFixed(2); // "1.77"

// 3. Katı Eşitlik (===) vs Gevşek Eşitlik (==)
"5" == 5;            // true  (Tipi çevirir - KÖTÜ)
"5" === 5;           // false (Değer + tip - DOĞRU)
```

Operatör	Görev	Örnek
+ - * /	Dört işlem	`10 / 2 → 5`
% (Modül)	Bölümden kalan	`8 % 3 → 2`
**** (Üs)**	Kuvvet alma	`5 ** 2 → 25`
=== / !==	Katı eşitlik	`5 === 5 → true`
&& / || / !	VE / VEYA / DEĞİL	`p > 50 && aktif`

BENZETME == sadece isme bakan dikkatsiz görevlidir; === ise hem isme hem TC kimlik numarasına (veri tipine) bakan sıkı güvenliktir.

METİNLERLE ÇALIŞMAK (STRINGS)

Şablon Değişmezleri ve Kullanışlı Dize Metotları

```
const rol   = "Geliştirici";
// Ters tırnak (`) ile Şablon Değişmezi
const mesaj = `Hoş geldin ${rol}, puan: ${45 * 2}`;

const metin = "mozilla";
metin.length;                 // 7     (karakter sayısı)
metin[0];                     // "m"   (ilk harf, 0'dan başlar)
metin.includes("zilla");      // true  (alt dize var mı?)
metin.startsWith("moz");      // true  (bununla mı başlıyor?)
metin.slice(1, 4);            // "ozi" (1'den 4'e kadar kes)
metin.toUpperCase();          // "MOZILLA"
metin.replace("moz", "van"); // "vanilla"
```

Şablon Değişmezi (`` `...${deger}` ``)

+ ile metin birleştirme derdini bitirir ve çok satırlı metinleri korur.

Arama: includes() ve indexOf()

`includes()` var mı diye bakar; `indexOf()` konumunu verir (yoksa `-1` döner).

Kesme ve Değiştirme

`slice(baş, son)` parça çıkarır; `replaceAll()` tüm eşleşmeleri değiştirir.

BENZETME Şablon değişmezi (`${}`), boşluk doldurmalı hazır dilekçe formu gibidir: sabit metin durur, sadece boşluğa değişken yerleşir.

ÇOKLU VERİ LİSTELERİ

Diziler ve Modern Dizi Metotları (map, filter)

```
const sehirler = ["Sivas", "Ankara"];
sehirler.push("İstanbul");    // Sona ekler
sehirler.pop();               // Sondan çıkarır
sehirler.unshift("İzmir");    // Başa ekler
sehirler.shift();             // Baştan çıkarır

const sayilar = [1, 2, 3, 4];
// map(): Her öğeyi dönüştürüp YENİ dizi döner
const ikiKati  = sayilar.map(x ⇒ x * 2);     // [2,
4, 6, 8]

// filter(): Şartı geçenlerle YENİ dizi döner
const buyukler = sayilar.filter(x ⇒ x > 2);  // [3,
4]

"Elma,Armut".split(",");     // ["Elma", "Armut"]
```

Sıfır Tabanlı Dizin (Zero-Indexed)

İlk eleman `dizi[0]`, son eleman `dizi[dizi.length - 1]` ile alınır.

map() vs filter()

`map()` her elemanı değiştirir (sayı değişmez); `filter()` şarta uymayanları eler.

split() ve join()

`split()` metni diziye böler; `join()` diziyi tekrar metne birleştirir.

BENZETME Dizi numaralı vagonları olan bir tren gibidir (0. vagondan başlar). `filter()` ise elek gibidir: sadece şarta uyan iri taşları üstte bırakır.

AKIŞ KONTROLÜ VE TEKRARLAYAN İŞLER

Koşullu İfadeler (if, switch) ve Döngüler (for, while)

```
// 1. Karar Verme: if/else ve Üçlü Operatör
const not = 75;
if (not ⩾ 85) {
  console.log("Pekiyi");
} else if (not ⩾ 50) {
  console.log("Geçti");
} else {
  console.log("Kaldı");
}
// Üçlü (Ternary): koşul ? doğru : yanlış
const sonuc = not ⩾ 50 ? "Başarılı" : "Kaldı";
```

```
// 2. Döngüler: for...of ve Klasik for
const dersler = ["HTML", "CSS", "JS"];
// Koleksiyonlar için en güvenli yol: for...of
for (const ders of dersler) {
  console.log(ders);
}
// Klasik sayaçlı for döngüsü
for (let i = 1; i ⩽ 5; i++) {
  if (i ⩵ 3) continue; // 3'ü atla
  if (i ⩵ 5) break;    // Döngüden çık
}
```

Falsy (Yanlışsı) Değerler

`false`, `0`, `""`, `null`, `undefined` ve `NaN` dışındaki her şey `true` sayılır.

break vs continue

`break` döngüyü anında bitirir; `continue` sadece o turu atlayıp sıradakine geçer.

BENZETME `if/else` yol ayrımındaki yön tabelasıdır; döngü ise koşu pistinde bitiş düdüğü çalana kadar tur atmaktır.

YENİDEN KULLANILABİLİR KOD BLOKLARI

Fonksiyonlar, Dönüş Değerleri (return) ve Kapsam (Scope)

```
// Varsayılan parametreli klasik fonksiyon
function selamla(rol = "Misafir") {
  return `Merhaba ${rol}!`; // Sonucu dışarı döndürür
}
const mesaj = selamla("Kullanıcı");

// Ok Fonksiyonu (Arrow Function) - Kısa ve modern
const kareAl = (x) ⇒ x * x;
kareAl(5); // 25

// Kapsam (Scope)
const binaBahcesi = "Herkes görür (Global)";
function daire() {
  const ozelOda = "Sadece içeriden erişilir (Local)";
}
```

Parametre vs Argüman

Tanımdaki değişken **parametre** (`rol`), çağrılırken gönderilen gerçek değer **argümandır** (`"Kullanıcı"`).

Dönüş Değeri (return)

Hesaplanan sonucu çağrıldığı yere teslim eder; `return` yoksa `undefined` döner.

Fonksiyon Kapsamı (Scope)

İçeride tanımlanan değişken dışarıdan okunamaz; isim çakışmalarını önler.

BENZETME Fonksiyon mutfak robotu gibidir: malzemeyi (parametre) atarsın, sana yemeği (`return`) verir. Kapsam ise apartman dairesidir: komşu senin odana giremez.

GERÇEK DÜNYAYI KODDA MODELLEMEK

JavaScript Nesne Temelleri, "this" ve Yapıcılar

```
const araba = {
  marka: "Togg",      // Özellik (Property)
  model: "T10X",
  bilgiVer() {        // Metot (Method)
    console.log(`Araç: ${this.marka} ${this.model}`);
  }
};
araba.marka;          // Nokta gösterimi: "Togg"
araba["model"];       // Köşeli parantez: "T10X"
araba.bilgiVer();     // "Araç: Togg T10X"

// Yapıcı (Constructor) / Sınıf ile çoklu üretim
class Cihaz {
  constructor(tur) { this.tur = tur; }
}
const yeni = new Cihaz("Dizüstü Bilgisayar");
```

Özellik (Property) ve Metot

Nesne içindeki değişkenlere **özellik**, fonksiyonlara ise **metot** denir.

"this" Anahtar Sözcüğü

Metodun o anda üzerinde çalıştığı nesnenin kendisini işaret eder.

Yapıcı (Constructor) ve new

Aynı kodu tekrar yazmadan tek kalıptan yüzlerce nesne üretmeyi sağlar.

BENZETME Nesne değişmezi tek tek elde yapılan kurabiyedir; Yapıcı (Constructor/Class) ise kurabiye kalıbıdır: tek kalıpla yüzlerce kurabiye basarsın.

BELGE NESNE MODELİ (DOCUMENT OBJECT MODEL)

DOM Ağacı: Öğeleri Seçmek, Oluşturmak ve Silmek

```javascript
// 1. Seçme (CSS Seçicisiyle)
const kutu = document.querySelector(".kutu");

// 2. Yeni düğüm (node) oluşturma ve metin ekleme
const yeniP = document.createElement("p");
yeniP.textContent = "Dinamik paragraf eklendi!";

// 3. Sınıf ekleme ve sayfaya yerleştirme
yeniP.classList.add("vurgu");
kutu.appendChild(yeniP); // Alt öğe olarak ekler

// 4. Öğeyi silme
yeniP.remove();
```

Window, Navigator, Document

`Window` tarayıcı sekmesi, `Navigator` tarayıcı kimliği, `Document` ise yüklü HTML ağacıdır.

DOM Ağacı Düğümleri (Nodes)

Kök (Root), Ebeveyn (Parent), Çocuk (Child) ve Kardeş (Sibling) düğümlerden oluşur.

Güvenlik: textContent vs innerHTML

XSS saldırılarını önlemek için metin eklerken `innerHTML` yerine `textContent` kullan.

BENZETME DOM bir soy ağacı gibidir: JavaScript bu ağacın dallarına yeni yapraklar (`appendChild`) ekleyebilir veya kurumuş dalları (`remove`) budayabilir.

KULLANICI ETKİLEŞİMİ VE EVENT BUBBLING

Olay Dinleyicileri, Kabarcıklanma ve Yetkilendirme

```
const btn = document.querySelector("button");
// Önerilen yöntem: addEventListener
btn.addEventListener("click", (e) ⇒ {
  e.stopPropagation(); // Yukarı kabarcıklanmayı durdurur
  console.log("Tıklanan:", e.target);
});

// Olay Yetkilendirmesi (Event Delegation)
// 16 karoya tek tek dinleyici takmak yerine üst kutuya tak!
kapsayici.addEventListener("click", (e) ⇒ {
  e.target.style.backgroundColor = "gold";
});
```

Neden onclick="" Kullanılmaz?

HTML içine JS karıştırmak bakımı zorlaştırır; `addEventListener` çoklu dinleyiciye izin verir.

Kabarcıklanma (Event Bubbling)

İçteki düğmeye tıklanınca olay sırasıyla üst `div` ve `body` öğelerine yayılır.

e.target vs e.currentTarget

`target` ilk tıklanan öğedir; `currentTarget` dinleyicinin bağlı olduğu üst öğedir.

BENZETME Olay dinleyici kapı zili gibidir: basınca çalışır. Kabarcıklanma ise suyun dibinden çıkan hava kabarcığının yüzeye (üst öğelere) doğru yükselmesidir.

SAYFAYI DONDURMADAN ÇALIŞMAK

Senkron ve Asenkron Mimari Karşılaştırması

Senkron (Synchronous – Bloklayan)

- **Sıralı İlerleme:** Biri bitmeden diğeri asla başlamaz.
- **Sorun:** Ağır bir işlem tüm sayfayı ve tıklamaları dondurur.
- **Garson Örneği:** Siparişi mutfağa verip yemek pişene kadar mutfak kapısında bekleyen garson.

Asenkron (Asynchronous – Bloklamayan)

- **Arka Plan:** Uzun iş başlatılır, sonucu beklenmeden yola devam edilir.
- **Avantaj:** Sayfa akıcı kalır; iş bitince callback/Promise haber verir.
- **Garson Örneği:** Siparişi mutfağa iletip hemen diğer masalara bakan, zil çalınca servisi yapan garson.

```
console.log("1. Sipariş alındı");
setTimeout(() ⇒ console.log("2. Yemek hazır (2 sn sonra)"), 2000);
console.log("3. Diğer masaya geçildi");   // Çıktı Sırası: 1 → 3 → 2
```

BENZETME Asenkron mimari çamaşır makinesini çalıştırıp başında beklemek yerine o yıkarken salonda çay içmektir; makine bitince ötüp haber verir.

JAVASCRIPT'İN KALBİ NASIL ATAR?

Call Stack, Web API'ler, Kuyruklar ve Event Loop

1. Call Stack (Yığın)

Şu an çalışan fonksiyonların yığınıdır. LIFO (Son giren ilk çıkar) kuralıyla işler.

2. Web API'ler

`setTimeout`, `fetch` ve DOM olaylarını tarayıcı arka planda yürütür.

3. Görev Kuyrukları

Arka planda işi biten callback'lerin Stack'e dönmek için sıra beklediği yerdir.

4. Event Loop

Stack boşaldığı anda kuyruktan işi alıp Stack'e taşıyan sonsuz nöbetçi döngüdür.

Call Stack (LIFO) ve Stack Overflow

`a()` → `b()` → `c()` çağrıldığında üst üste biner; en son giren `c()` ilk çıkar. Çıkış koşulsuz özyineleme yığını taşırır (Stack Overflow).

5 Adımda Çalışma Akışı

1. Kod Stack'e girer → 2. Web API devralır → 3. Kuyruğa geçer → 4. Stack boşalır → 5. Event Loop işi Stack'e taşır.

BENZETME Call Stack üst üste dizilen tabaklar gibidir (en üstteki ilk alınır). Event Loop ise mutfak tezgâhındaki hazır yemeği boşalan masaya taşıyan şeftir.

KUYRUK ÖNCELİKLERİ VE ADIM ADIM ÇIKTI ANALİZİ

Microtask vs Task (Macrotask) Karşılaştırması

Microtask Kuyruğu (VIP Öncelikli)

`Promise.then`, `catch`, `await` sonrası kodlar. Stack boşaldığında **hepsi bitene kadar** çalışır.

Task / Macrotask Kuyruğu (Normal Sıra)

`setTimeout`, `setInterval`, DOM tıklamaları. Microtask tamamen boşaldıktan sonra her turda **bir adet** alınır.

```
// Klasik Mülakat & Sınav Sorusu: Çıktı Ne Olur?
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");

// KESİN ÇIKTI: 1, 4, 3, 2
```

Neden 1 → 4 → 3 → 2?

1 ve 4 senkrondur, hemen Stack'te çalışır. **3** Promise olduğu için VIP (Microtask) kuyruğuna girer. **2** ise 0 sn olsa bile Task kuyruğunda en son sırayı bekler.

BENZETME Banka gişesi gibidir: Gişedeki mevcut işlem (Senkron) bitince, VIP müşterilerin (Microtask) tamamı normal sıra fişi alanlardan (Task) önce hizmet alır.

SUNUCUDAN VERİ ÇEKMEK (AJAX & FETCH API)

Fetch API: Callback, Promise ve async/await Evrimi

1. Eski: Callback

```
veriAl(url, (hata, veri) ⇒ {
  if (hata) return;
  isle(veri, (sonuc) ⇒ {
    console.log(sonuc);
  });
});
// İç içe geçer (Callback Hell)
```

2. Promise Zinciri (.then)

```
fetch("urunler.json")
  .then(r ⇒ {
    if (!r.ok) throw
Error(r.status);
    return r.json();
  })
  .then(veri ⇒ goster(veri))
  .catch(err ⇒
console.error(err));
```

3. Modern: async / await

```
async function yukle() {
  const r    = await
fetch("urun.json");
  const veri = await r.json();
  goster(veri);
}
// Senkron kod gibi düz okunur!
```

Neden Sayfayı Yenilemeyiz?

Tüm siteyi baştan indirmek yerine sadece değişen veriyi (JSON) çekerek hız ve bant genişliği kazanırız.

Fetch vs Eski XMLHttpRequest

`fetch()` Promise tabanlıdır; `response.json()`, `text()` ve `blob()` ile veriyi kolayca çözer.

BENZETME `fetch()` internetten kargo sipariş etmektir; döndürdüğü `Promise` ise kargo takip numarasıdır: paket kapıya gelince `.then()` veya `await` ile kutuyu açarsın.

VERİ İLETİŞİMİ VE MODERN SÖZDİZİMİ KISALTMALARI

JSON Dönüşümleri ve JavaScript'in Özel Karakterleri

⇒ Ok Fonksiyonu

Fonksiyonu tek satırda kısa yazar, callback'leri sadeleştirir.

```
const kare = x ⇒ x * x; // kısa
```

... Yayma (Spread)

Diziyi veya nesneyi açıp kopyalar, yeni elemanlarla birleştirir.

```
const yeni = [...dizi, 4]; // kopya
```

?? Null Birleştirme

Yalnızca `null` veya `undefined` ise varsayılan değeri atar (0'ı korur).

```
const adet = x ?? 0; // yedek değer
```

`${}` Şablon Metin

Ters tırnak içine doğrudan değişken ve hesaplama gömer.

```
`Konu: ${baslik}, Süre: ${dk}`
```

?. Opsiyonel Zincir

Özellik yoksa programı çökertmez, güvenle `undefined` döner.

```
veri?.adres?.sehir // güvenli
```

JSON Parse & Stringify

Çift tırnak zorunludur; metni nesneye, nesneyi metne çevirir.

```
JSON.parse(JSON.stringify(o))
```

BENZETME Özel operatörler telgraf kısaltmaları gibidir: az karakterle çok iş yapar. `JSON.stringify` ise mobilyayı kargolamak için düz koliye paketlemektir.

SORUN GİDERME VE SAVUNMACI KODLAMA

Hata Türleri, DevTools Debugger ve try...catch

1. Sözdizimi (SyntaxError)

Eksik parantez, tırnak veya yazım hatası (`addeventListener`); kod hiç çalışmaz.

2. Çalışma Zamanı (Runtime)

Kod başlar ama olmayan öğeye erişirken çöker (`lowOrHi is null`).

3. Mantık Hatası (Logic)

Hata mesajı vermez ama yanlış sonuç üretir (== yerine = yazmak gibi).

```
// Savunmacı kontrol ve özel hata fırlatma
function metreyeCevir(inc) {
  if (typeof inc !== "number" || Number.isNaN(inc)) {
    throw new Error("Lütfen geçerli bir sayı girin!");
  }
  return (inc * 2.54) / 100;
}
try {
  console.log(metreyeCevir("yetmiş"));
} catch (hata) {
  console.error("Yakalanan Hata:", hata.message);
}
```

Araçlar: Breakpoint ve ESLint

DevTools **Debugger** sekmesinde satıra kesme noktası (breakpoint) koyup değişkenleri canlı izleriz. **ESLint** ise hataları daha kodu yazarken yakalar.

BENZETME `try ... catch` sirkteki güvenlik ağı gibidir: cambaz (kod) düşse bile (`throw`) yere çakılmaz, ağ onu yakalar ve gösteri çökmeden devam eder.

NEDEN ÇERÇEVE (FRAMEWORK) KULLANIRIZ?

Kütüphane vs Çerçeve, DOM Zorluğu ve 4 Büyükler

Sorun: Sade JS ile DOM Güncellemek

Uygulama durumu (state) her değiştiğinde `createElement` ve `appendChild` ile ekranı elle eşitlemek 30+ satır spagetti kod üretir.

Çözüm: Bildirimsel (Declarative) UI

Çerçevelere arayüzün **nasıl görünmesi gerektiğini** söyleriz; DOM güncellemesini perde arkasında çerçeve otomatik yapar.

React (2013)

Facebook geliştirdi. UI kütüphanesidir (ReactDOM ile web'de çalışır). JSX kullanır.

Vue (2014)

Evan You geliştirdi. HTML şablonlarını `v-for` gibi özniteliklerle genişletir.

Angular (2016)

Google geliştirdi. Tam kapsamlıdır; varsayılan olarak TypeScript kullanır.

Ember (2011)

Köklü ve katı kurallı mimari sunar; Handlebars şablon dilini kullanır.

BENZETME Kütüphane (Library) elindeki çekiçtir, istediğin zaman sen çağırırsın; Çerçeve (Framework) ise hazır fabrika bandıdır, kuralları o koyar ve senin kodunu o çağırır.

ORTAK MİMARİ KAVRAMLAR

DSL (JSX, TypeScript), Sanal DOM ve Yaşam Döngüsü

```
// JSX ve TypeScript (Alana Özgü Diller - DSL)
// JSX: JavaScript içinde HTML benzeri yazım
const konu   = "Dünya";
const baslik = Merhaba {konu}!;
// Derlenince: React.createElement("h1", null, ...)

// TypeScript: Tip güvenliği eklenmiş JS üst kümesi
function topla(a: number, b: number) {
  return a + b; // Metin gelirse derlemede uyarır!
}
```

Sanal DOM (Virtual DOM)

DOM'un bellekteki kopyasıdır. Değişiklikte gerçek DOM ile farkı (diff) bulunur ve sadece değişen düğüm güncellenir.

Yaşam Döngüsü (Lifecycle)

Bileşenin ekrana bağlanması (mount), işlenmesi (render) ve kaldırılması (unmount) evreleridir.

Prop Delme ve Bağımlılık Enjeksiyonu

Veriyi katman katman indirmek yerine Context API ile doğrudan hedefe ulaştırır.

BENZETME Sanal DOM mimarın eskiz kağıdıdır: bütün binayı yıkıp yeniden yapmak yerine önce kağıtta farkı (diff) bulur, binada sadece o pencereyi değiştirir.

REACT'E BAŞLARKEN VE COMPONENT MİMARİSİ

Vite Kurulumu, JSX Kuralları, Bileşenler ve Props

```
// 1. Vite ile Proje Başlatma:
// npm create vite@latest moz-todo-react -- --template react

// 2. Bileşen Tanımı (PascalCase zorunlu)
function Todo(props) {
  return (


      {props.name}

  );
}
// 3. Listeyi map() ve benzersiz key ile işleme
const taskList = tasks.map(t => (

));
```

Özel JSX Öznitelikleri

JS ayrılmış kelimeleriyle çakışmamak için `class` yerine `className`, `for` yerine `htmlFor` yazılır.

Props (Özellikler)

Üst bileşenden alt bileşene tek yönlü veri taşır; salt okunurdur (değiştirilemez).

Benzersiz key Zorunluluğu

`map()` ile üretilen her öğeye React'in takip edebilmesi için benzersiz `key` verilir.

BENZETME Bileşenler (Components) Lego parçaları gibidir: `Todo`, `Form` ve `FilterButton` parçalarını ayrı yapıp `App` içinde birleştirirsin.

REACT ETKİLEŞİMİ VE KANCALAR (HOOKS)

useState, Callback Props, Koşullu İşleme ve Odak Yönetimi

```jsx
import { useState, useRef, useEffect } from "react";

function SayacVeOdak() {
  // useState: [değer, güncelleyiciFonksiyon]
  const [sayac, setSayac]     = useState(0);
  const [duzenle, setDuzenle] = useState(false);
  const inputRef              = useRef(null); // DOM hedefi

  // useEffect: Render bittikten hemen sonra çalışır
  useEffect(() => {
    if (duzenle) inputRef.current.focus();
  }, [duzenle]);

  return (

      {duzenle ?  : {sayac}}
       setSayac(sayac + 1)}>Artır

  );
}
      {duzenle ?  : {sayac}}
       setSayac(sayac + 1)}>Artır
```

State (Durum) ve Callback Props

`useState` bileşenin hafızasıdır. Alt bileşenden üst bileşene veri göndermek için fonksiyonlar prop olarak geçirilir.

Koşullu İşleme (Conditional Render)

`isEditing ? editingTemplate : viewTemplate` ile ekranda hangi şablonun görüneceği seçilir.

useRef ve useEffect ile Erişilebilirlik

Şablon değiştiğinde kaybolan klavye odağını `ref.current.focus()` ile doğru kutuya taşırız.

BENZETME Props sana dışarıdan verilen kimlik kartıdır (değiştiremezsin); State ise kendi not defterindir (`useState` ile yazar, siler, güncellersin).

HERKES İÇİN WEB (A11Y & WCAG İLKELERİ)

Web Erişilebilirliği Nedir ve Kimleri Kapsar?

1. Görme Engelleri
Körlük, az görme, renk körlüğü. Ekran okuyucular (NVDA, VoiceOver) ve büyüteçler kullanılır.

2. İşitme Engelleri
Sağırlık ve işitme kaybı. Videolarda altyazı () ve metin dökümleri şarttır.

3. Motor / Fiziksel
Fare kullanamama, felç, titreme. Tam klavye kontrolü ve sesli komut sistemleri gerekir.

4. Bilişsel Engeller
Disleksi, dikkat eksikliği. Sade tasarım, net başlık düzeni ve tutarlı gezinme gerekir.

Neden Herkese Faydalıdır?
Kolunu kıran biri geçici olarak fare kullanamaz; gürültülü otobüste video izleyen herkes altyazıya ihtiyaç duyar. Semantik kod SEO ve mobil hızı da artırır.

WCAG Kılavuzları ve Yasal Zorunluluk
W3C WCAG standartları (Algılanabilir, Çalıştırılabilir, Anlaşılabilir, Sağlam) pek çok ülkede yasal zorunluluktur.

BENZETME Kaldırım rampaları tekerlekli sandalye için yapılmıştır ama bebek arabası süren ebeveynlerin ve valiz çeken yolcuların da hayatını kurtarır (Curb-Cut Effect).

DOĞRU İŞ İÇİN DOĞRU ÖĞE

Semantik HTML, Klavye Erişimi, CSS ve JS En İyi Uygulamaları

```
Gönder

Gönder
<button type="button">Gönderbutton>


<label for="eposta">E-posta Adresiniz:label>
<input type="email" id="eposta" required />


<img src="sus-cizgi.png" alt="" />
```

Semantik HTML = Bedava Klavye Desteği

, ve öğeleri Tab ile odaklanmayı ve Enter ile tetiklenmeyi yerleşik sunar.

CSS: Kontrast ve Odak Çizgisi

Yüksek renk kontrastı kullan; `outline: none` ile klavye odak halkasını asla tamamen yok etme!

JS: Fareye Bağımlı Olma

Sadece `mouseover` kullanma; klavye için `focus` ve `keydown` olaylarını da destekle.

BENZETME elementini CSS ile boyayıp düğme yapmak, duvara kapı resmi çizmeye benzer: uzaktan kapı gibi görünür ama içinden kimse geçemez.

WAI-ARIA TEMELLERİ, MULTİMEDYA VE ARAÇLAR

Roller, Durumlar, Canlı Bölgeler ve Ekran Okuyucu Testi

```
<button aria-pressed="true">Tümübutton>
<button aria-expanded="false">Menüyü Açbutton>


<div aria-live="polite" role="alert">
  Sepete 1 ürün eklendi!
div>


<div tabindex="0">Klavyeyle Tab sırasına girerdiv>
<h2 tabindex="-1">Sadece JS focus() ile odaklanırh2>
```

WAI-ARIA'nın 3 Sütunu

Roller (`role="tablist"` ne olduğunu), **Özellikler** (`aria-labelledby` anlamını) ve **Durumlar** (`aria-disabled` anlık halini) bildirir.

Multimedya ve Mobil Erişilebilirlik

Videolara WebVTT () altyazı eklenir; mobilde VoiceOver (iOS) ve TalkBack (Android) dokunma hareketleriyle çalışır.

Denetim Araçları (Auditing)

WAVE, Lighthouse ve ekran okuyucu kısayolları ile sayfa test edilir.

BENZETME WAI-ARIA müzedeki sesli rehber kulaklığı gibidir: görenler vitrine bakar, görmeyenlere ise `aria-live` sahnede ne değiştiğini anında kulağına fısıldar.

SINAV VE MÜLAKAT ÖZET TABLOSU

Özet: 3 Modülde Neler Öğrendik?

1. JS Temelleri (Modül 1)

`const/let`, katı eşitlik (`===`), şablon dizeler, `map/filter`, fonksiyon kapsamı ve nesneler.

2. DOM ve Olaylar (Modül 1)

`querySelector`, `addEventListener`, olay kabarcıklanması (bubbling) ve yetkilendirme.

3. Event Loop & Async (Modül 1)

Call Stack (LIFO), Web API, Microtask (Promise) önceliği, `fetch()`, JSON ve `try...catch`.

4. Çerçeve Mimarisi (Modül 2)

Bildirimsel UI, React/Vue/Angular/Ember, JSX, TypeScript ve Sanal DOM fark (diff) mantığı.

5. React & Hooks (Modül 2)

Bileşenler, Props, `useState` ile durum, geri çağırma prop'ları ve `useRef/useEffect` ile odak.

6. Erişilebilirlik (Modül 3)

WCAG, semantik HTML, tam klavye desteği, WAI-ARIA (`role`, `aria-live`) ve ekran okuyucular.

BENZETME Event Loop bir orkestra şefi, React sahneyi kuran dekor ekibi, Erişilebilirlik ise konser salonuna herkesin rahatça girebilmesini sağlayan mimaridir.

Dinlediğiniz İçin Teşekkürler!

Sorularınızı bekliyoruz.

Sunan: Ayşe Nurcihan Akbaş | Grup: Altay

Kaynak: MDN Web Docs > Learn Web Development > Core Modules

```
// kapanis.js — Oturum sonu
async function sunumuTamamla() {
  await modulleriAnlat([1, 2, 3]);
  console.log("Dinlediğiniz için teşekkürler!");

  // Soru - cevap dinleyicisi aktif
  window.addEventListener("soru", (e) => {
    cevapla(e.detay);
  });
}

sunumuTamamla();
```

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir (CC-BY-SA).